# Design Document: Alliant – Getting To Know You

---

## Overview

*Alliant: Getting To Know You* is a web-based OSINT (Open Source Intelligence) tool that collects, organizes, and summarizes publicly available information about individuals. Using AI-generated summaries and keyword-driven scraping, it helps analysts, investigators, and researchers build concise digital profiles based on names, usernames, websites, and social presence.

---

## Objectives

- Collect data relevant to a person using filters such as name, location, age, hobbies, etc.
- Retrieve profile images, recent activity/posts, and social media links.
- Summarize content using AI to deliver clear overviews.
- Organize information by category for frontend display and backend storage.
- Use environment variables to customize performance and security parameters.

---

## System Components

### 1. Frontend (React/Next.js)

- **Form Fields:**
    - `Name` (required)
    - `Company` (optional)
    - `Known Website` (optional)
    - `Username` (optional)

- **Main Folders:**
    - `/app/page`: Main logic for profile rendering.
    - `/components/advanced-search-bar`: User input.
    - `/search/page`: Renders search results.
    - `/components/`:
        - `AiSiteQuery`: Displays known sites.
        - `AutoScrapeSearch`: Displays AI-generated article summaries.
        - `GoogleImageSearch`: Handles image results.
        - `GoogleApiSearch`: Renders social media links.
        - `WhatsMyReport`: Displays username search results.
- **Features:**
    - Progress bar for task updates.
    - Stop button for long-running jobs.
    - Summarized results, photos, links, and profile metadata.

---

**2. Scraping & AI Summarization Scripts**

- `backend/finder.py`:
    - `google_search`: Gets social media URLs.
    - `google_image_search`: Gets image URLs.
    - `username_search`: Finds sites tied to usernames.

- `backend/backendwithAILinks.py`:
    - `scrape_website_text`: Scrapes main text data from websites.
    - `generate_summary/generate_summary2`: Uses a generative model and creates summaries from scraped text.
    - `get_google_links`: Finds articles to be used for the AI summaries.

---

**3. Backend (FastAPI)**

- Located at: `backend/backendwithAILinks.py`
- Responsibilities:
    - Connects to MongoDB.
    - Fetches/stores summaries and structured data.
    - Calls to generate the searches if not in database
    - Uses environment variables for:
        - `TIME_CUTOFF_DAYS`
        - `CORPUS_LIST_LIMIT`
        - `GENAI_API_KEY`
        - `GOOGLE_API_KEY`
        - `GOOGLE_CX`
        - `MONGO_URL`

**Dynamic Metadata Stored**:

```
{
  "timestamp": str(datetime.now()),
  "version": VERSION,
  "name": name,
  "Company": company,
  "images": [{"url": image, "isRelevant": False} for image in images],
```

"socials": [{"url": url, "isRelevant": False} for url in socials],

"auto_scrape": {"url": link, "summary": {"text": ai, "isRelevant": False}}

}

- **Conditional Fields**:

If `known_url` is provided:

"web_scrape": {

"url": {"text": url, "isRelevant": False},

"summary": {"text": summary[0], "isRelevant": False},

"marriage_status": {"text": summary[1], "isRelevant": False},

"interests": {"text": summary[2], "isRelevant": False},

"education": {"text": summary[3], "isRelevant": False}

}

If `username` is provided:

doc["userAccount"]: account["site_name"]: {"url": account["url"], "isRelevant": False}

---

## 4. Environment Configuration

Create a `.env` file inside the `backend/` folder with the following:

GENAI_API_KEY="Your-API-Key"

GOOGLE_API_KEY="Your-API-Key"

GOOGLE_CX="Your-CX-Key"

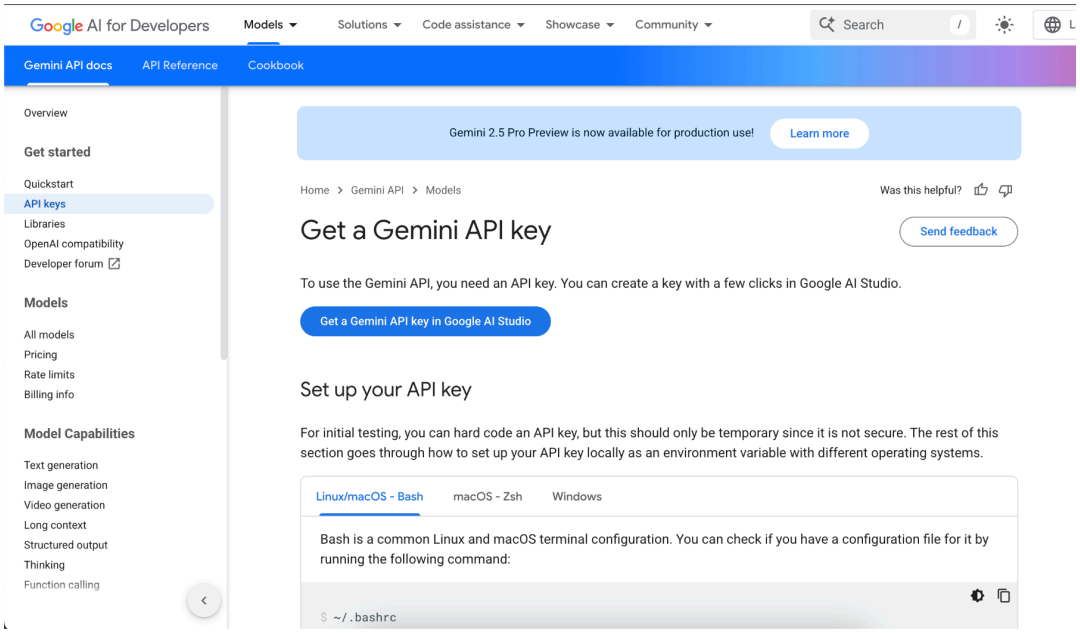
MONGO_URL="mongodb://127.0.0.1"

CORPUS_LIST_LIMIT="100"

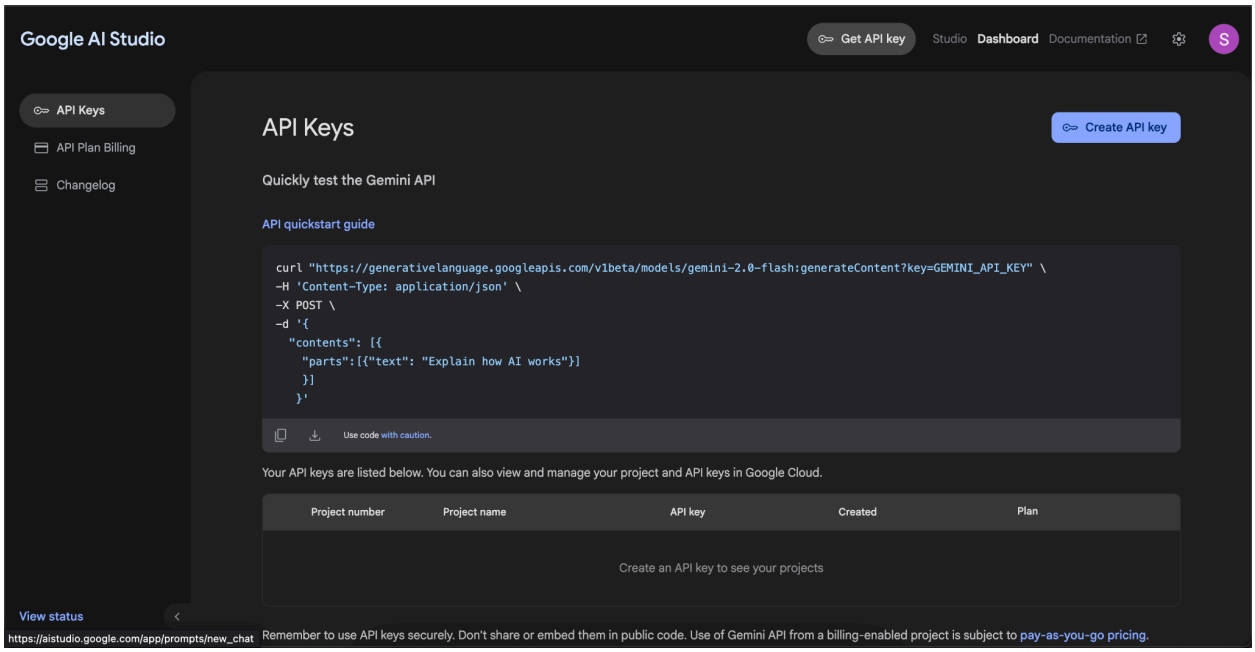
TIME_CUTOFF_DAYS="30"

# Instructions for Creating a GEMINI AI API Key

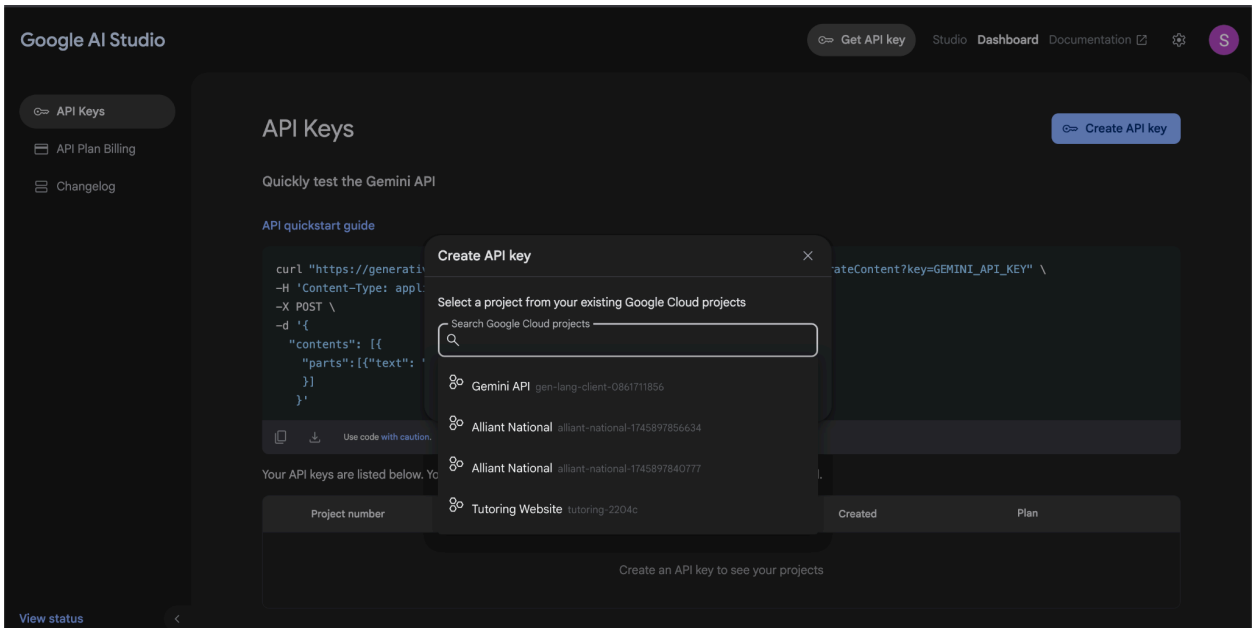
1. Visit https://ai.google.dev/gemini-api/docs/api-key
2. Click the **“Get a Gemini API key in Google AI Studio”** button

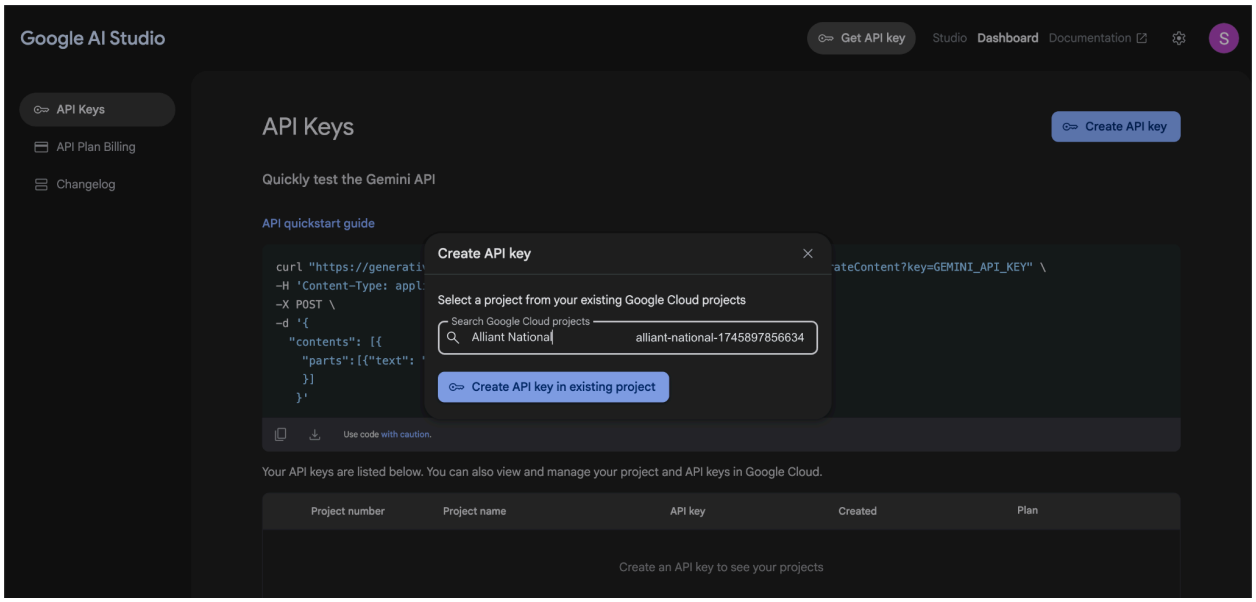



3. After being redirected here, click the **“Create API Key”** button

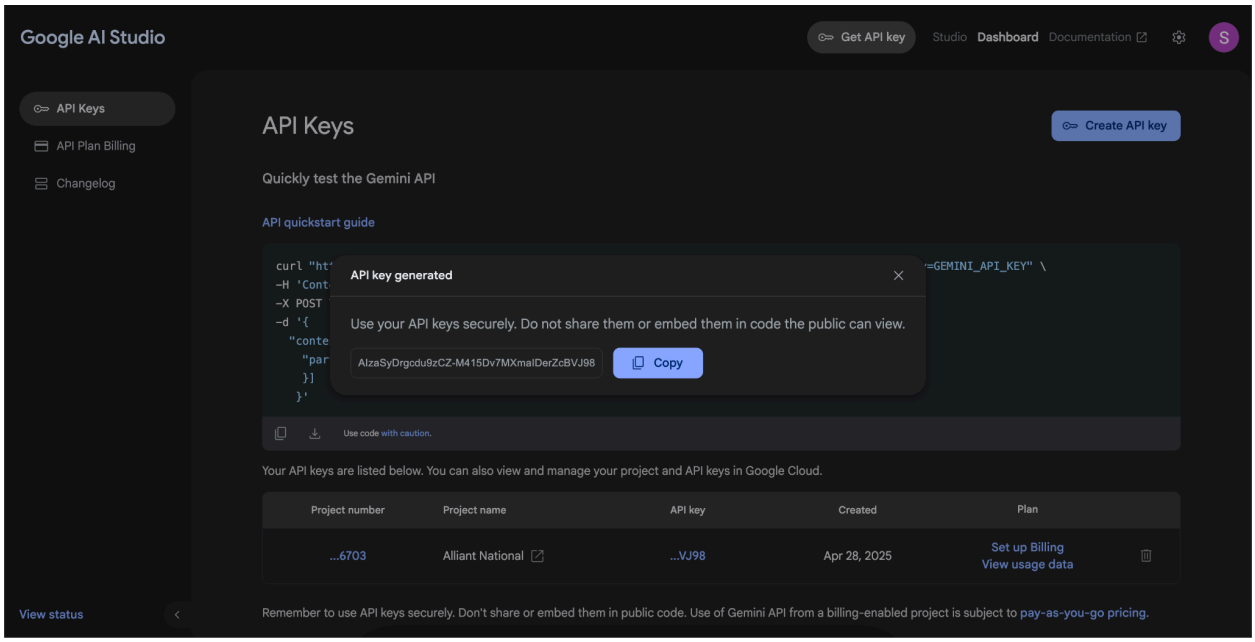


4. Click the project name.



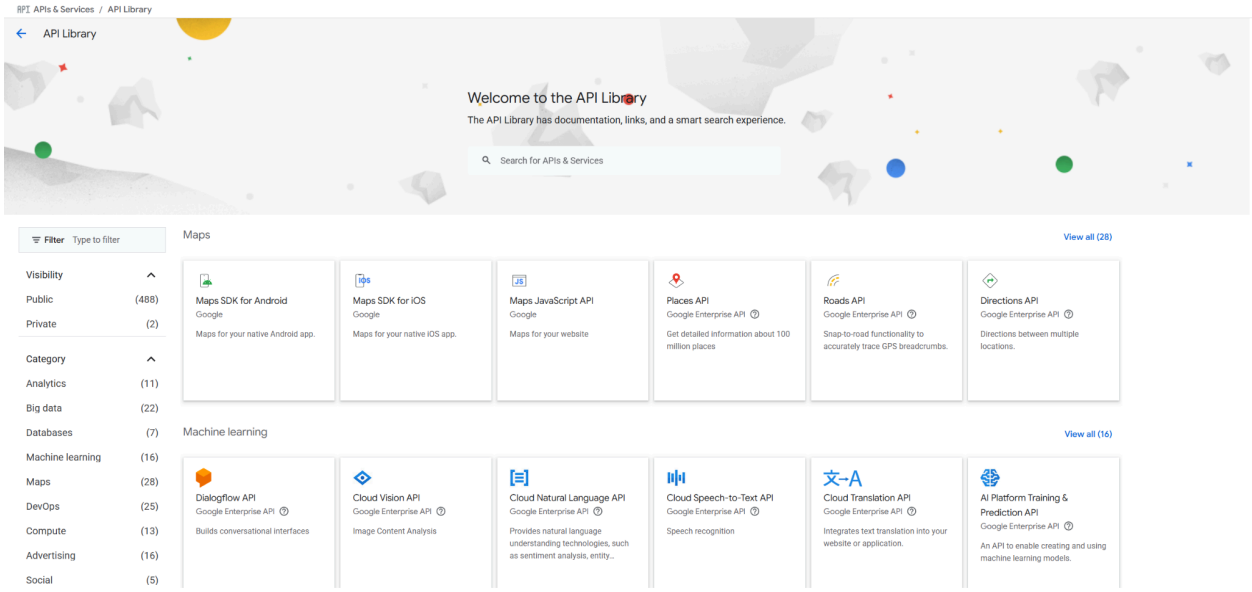



5. Once created, your API key will be displayed — copy and save it securely.

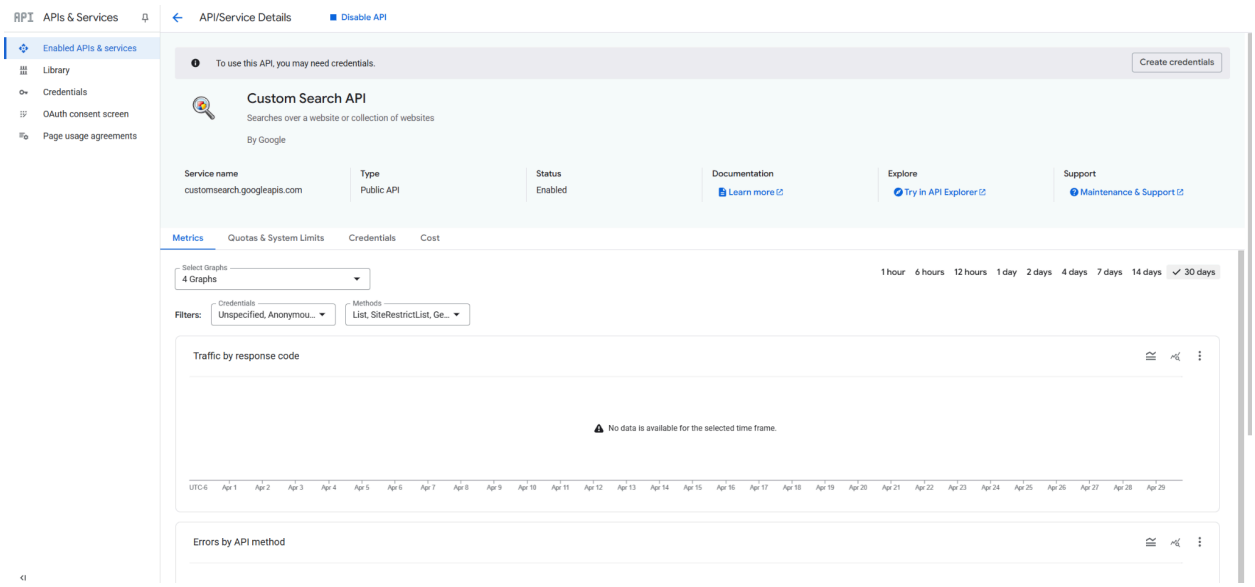



## Instructions for Creating a Google API Key

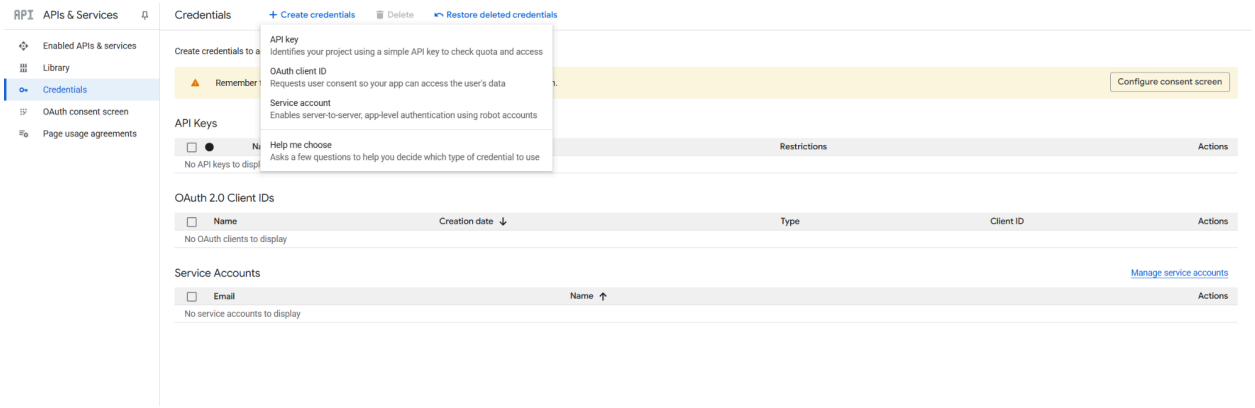
1. Visit https://console.cloud.google.com/
2. Click the "**Create or select a project**" button. From the pop up, hit "**New Project**" in the top right. Give it a name and hit create.
3. From the dashboard, on the left hand panel, hover over "**APIs & Services**" and click "**Library**."




4. Find the "**Custom Search API**" from the search bar and click to enable it.




5. From the Credentials tab under APIs & Services, hit "**Create credentials**" and "**API key**."

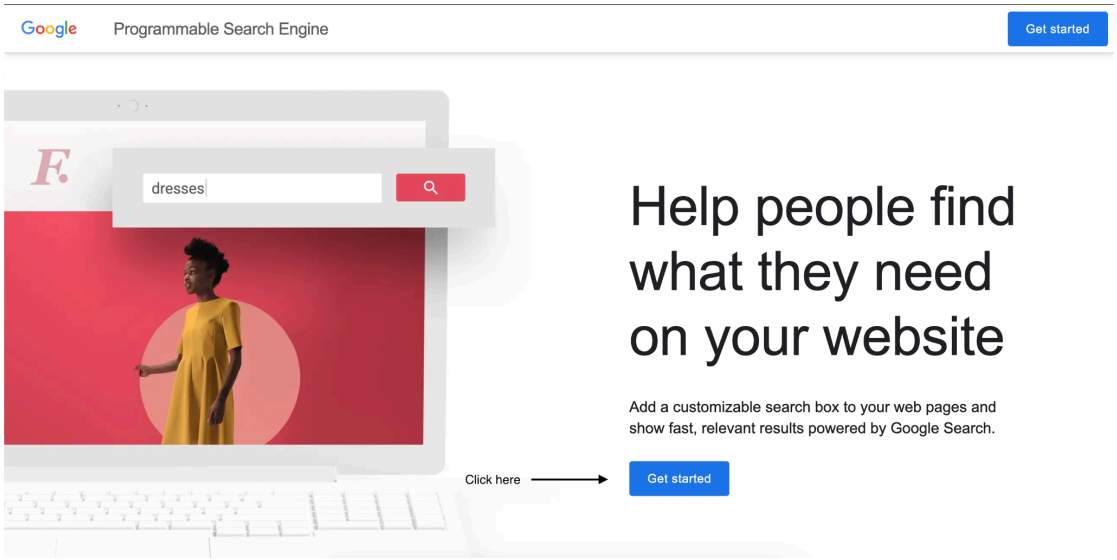



6. Once created, your API key will be displayed — copy and save it securely. This will be the GOOGLE_API_KEY variable inside the `.env` file.

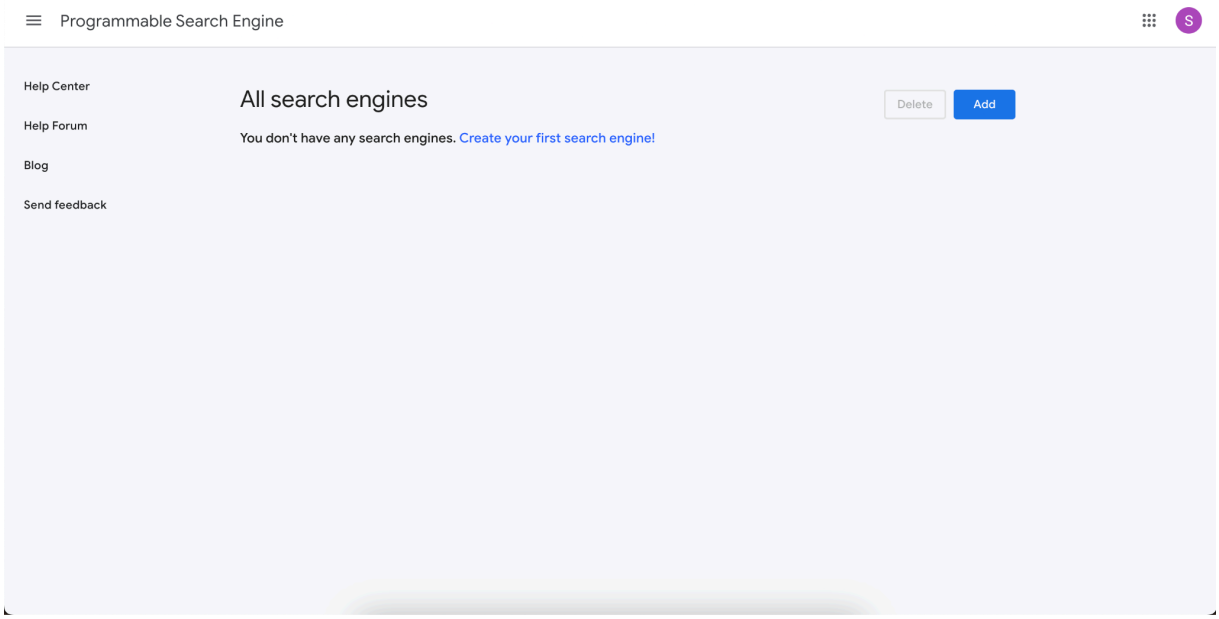
## Instructions for Creating a Google Search Engine ID (GOOGLE_CX)

**Visit the following URL:** https://programmablesearchengine.google.com/about/

1. Click the blue **Get Started** button.

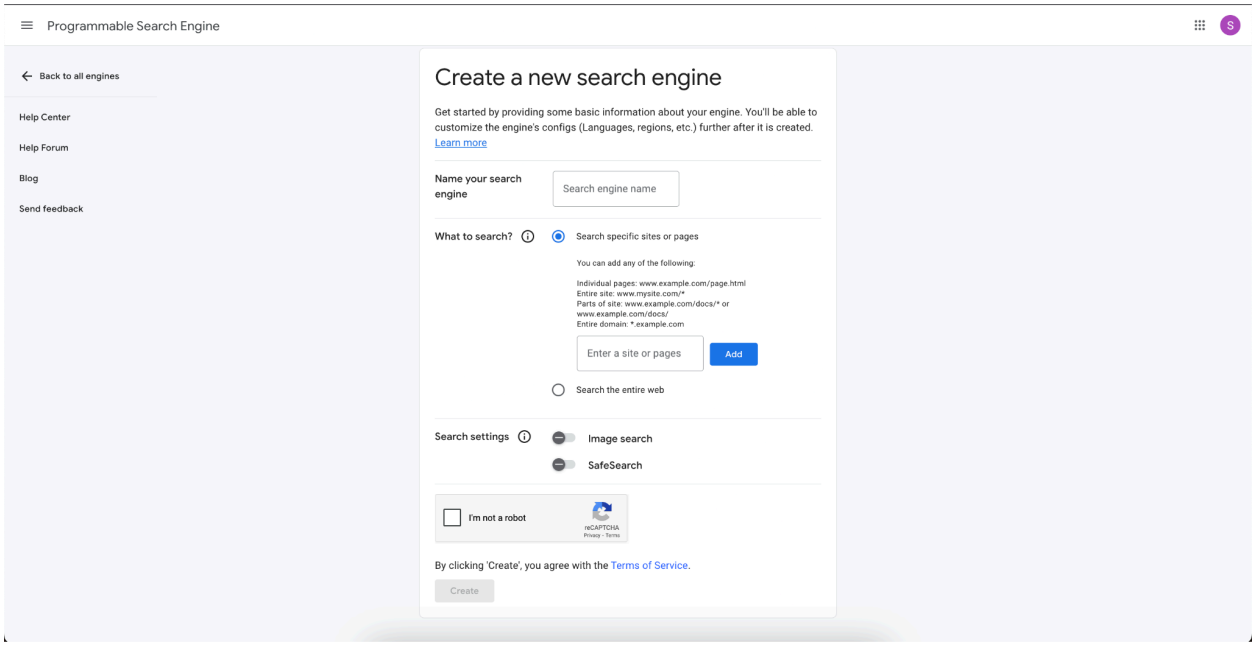



**2.** Click the blue **Add** button.



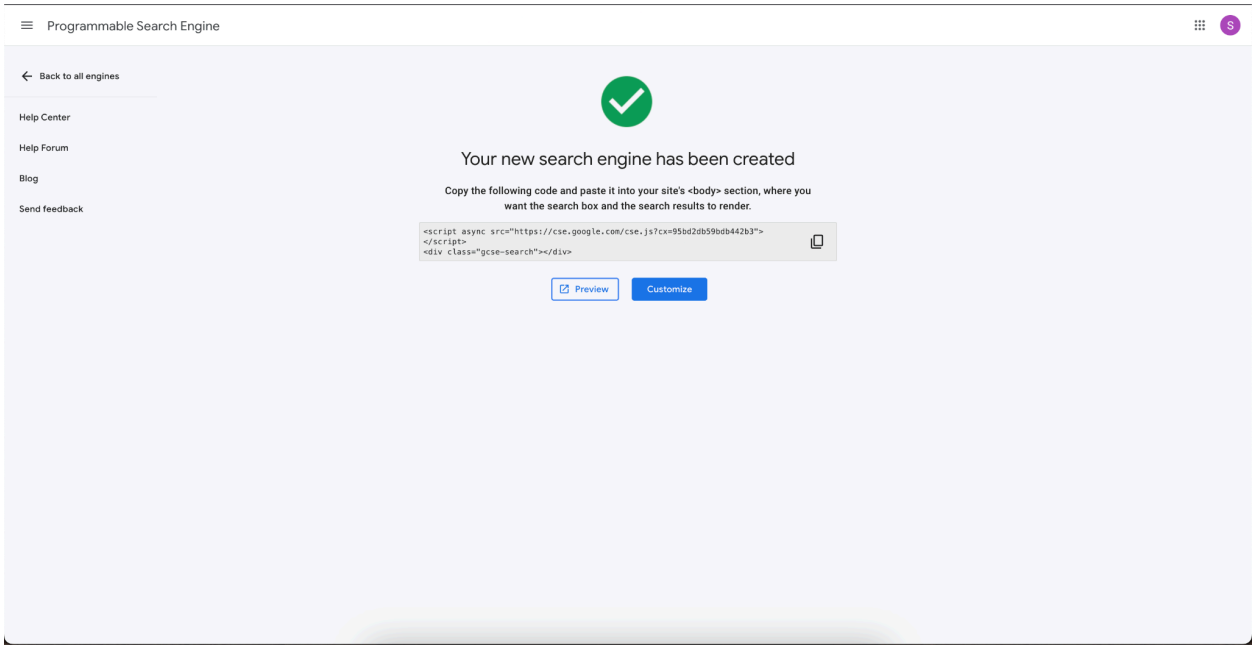

**3.** A menu will appear where you can enter a name for your search engine.

Make sure to select **"Search the entire web"** as the setting, then click **Create**.



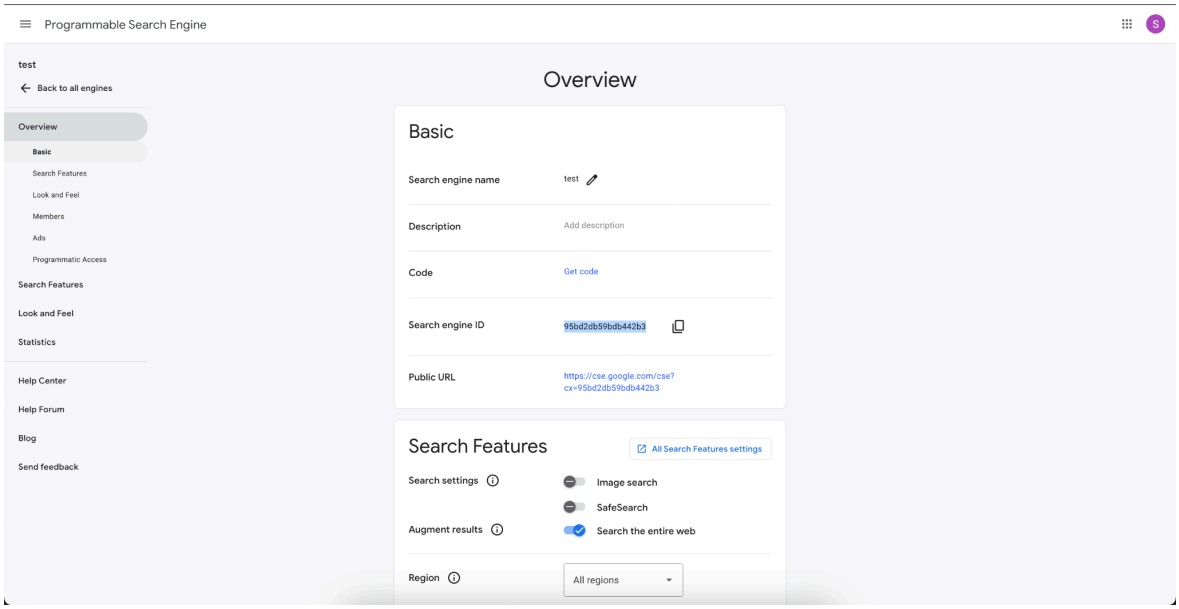

**4.** After creating your search engine, you will be taken to a new screen.

Click the blue **Customize** button to continue.




**5.** On the customization page, locate your **Search Engine ID**.

This ID will be used as the value for the `GOOGLE_CX` setting in your project.




---

## Database (MongoDB)

Stores structured JSON-like documents. Fields include:

- Name, Company, Age, Location
- Profile Images
- Keywords and relevant matches
- Summary paragraphs
- Timeline metadata

Routes include:

- `/corpus/find_info`: To run the command to find and store a new table entry.
- `/corpus/search`:To search the corpus with specific parameters.
- `/corpus/test`: Used for testing purposes.
- `/corpus`: To display all of the json in plain text.
- `/docs`: To interact with the auto setup frontend of the FastAPI.

---

## Installation

### Requirements

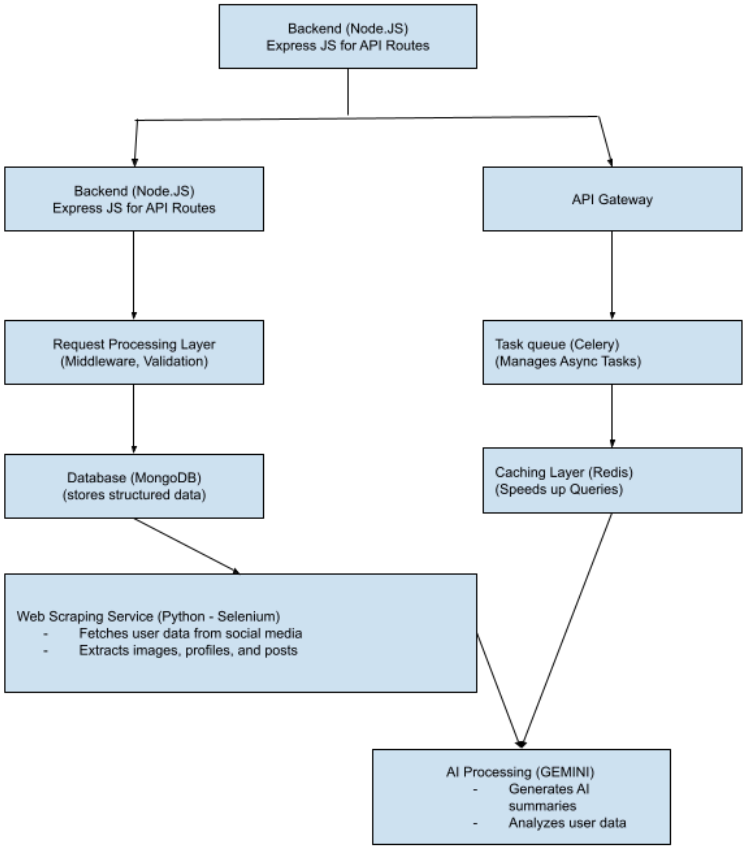
- Python 3.8+
- Node.js and npm (for frontend)
- MongoDB (local or cloud)
- `.env` file

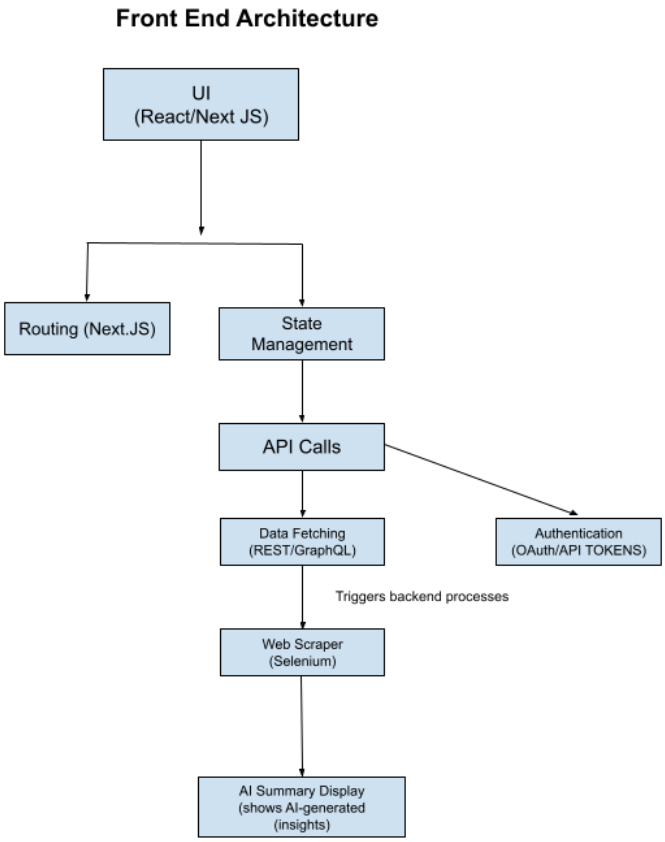
### Setup Instructions (local development):

1. **Backend**
    - Navigate to `backend/`
    - For dependencies: `pip install -r requirements.txt`
    - To Run Development: `fastapi dev backendwithAILinks.py`
2. **Frontend**
    - Navigate to `alliant-getting-to-know-you/`
    - For dependencies: `npm install`
    - To Run Development: `npm run dev`
3. **MongoDB**
    - Need it running locally as a service for the backend fastapi to work.

## Architecture Diagrams

# Backend Architecture






**Backend:**

- Node.js/Express (Frontend)
- Python microservices
- MongoDB, Celery (for async)
- AI: Gemini API
- (Legacy: Redis caching)

**Frontend:**

- React/Next.js
- REST APIs to backend

- Optional OAuth2

---

## Future Enhancements

- Containerization with Docker
- Retry logic for scrapers
- Admin dashboard for task tracking
- Emotion & intent detection in summaries
- Allow user to choose LLM (e.g., ChatGPT, Claude, Gemini)

---